DocuConverter Pro

Developer Handbook & Reference Manual

Django · MySQL · Python · HTML/CSS/JS

1. Project Overview

What is DocuConverter Pro?

DocuConverter Pro is an enterprise-grade, web-based document conversion suite built with Django and Python. Users can upload files through a browser and perform professional-quality conversions, compressions, merges, splits, and edits — all without installing any desktop software.

Project Goal

The project aims to solve core challenges of cross-format file operations while keeping a clean, permanent database history log. Utilizing a MySQL status dropdown system, conversions are never permanently deleted from phpMyAdmin, offering full control to administrative staff.

---

2. Features & Capabilities

Feature	Description
PDF ↔ Word	Convert PDFs to editable DOCX files and Word documents to pixel-perfect PDFs using Microsoft Word COM automation.
PDF ↔ Excel	Extract tabular data from PDFs into spreadsheets, or produce PDF reports from Excel files.
PDF ↔ PowerPoint	Convert presentation slides to PDF or rebuild PDFs as editable PPTX files.
Image to PDF	Bundle multiple JPEG/PNG images into a single merged PDF document.
JPEG ↔ PNG	Lossless image format conversion between JPEG and PNG.
Merge	Combine multiple PDFs, Word documents, or Excel files into a single output file.
Split	Extract specific pages or ranges from PDFs, Word, and Excel files.
Compress	Reduce file sizes for PDF, DOCX, XLSX, and PPTX documents without visible quality loss.
Word Editor	WYSIWYG in-browser editor for Word documents — edit content and export back to DOCX.
PDF Editor	Make text edits and highlights directly on PDF files without converting them.
Conversion History	Every successful conversion is saved with a status dropdown. History supports soft delete, restore, and clear.

---

3. Technology Stack

Package / Technology	Role
Python 3.14	Core programming language.
Django 6.0.6	Web framework — routing, ORM, settings, and template rendering.
MySQL (MariaDB 10.4)	Production database via XAMPP. Stores all conversion history permanently.
mysqlclient 2.2.8	High-performance C-binding driver connecting Django to MySQL.
HTML5 / CSS3 / JS	Frontend dashboard, theme toggle, sidebar history, and WYSIWYG word editor.
Microsoft Word COM	Win32 COM automation for pixel-perfect Word-to-PDF conversion on Windows.
PyMuPDF (fitz)	Fast C-wrapped PDF reading, text extraction, splitting, and merging.
pdf2docx	PDF-to-Word structural reconstruction engine.
python-docx	Read and write native DOCX files programmatically.
openpyxl	Excel (.xlsx) file reading, writing, merging, and splitting.
python-pptx	PowerPoint (.pptx) file processing and slide operations.
reportlab	Programmatic PDF generation and image-to-PDF bundling.
Pillow	Image processing: format conversion, resizing, and compression.

4. Project Folder Structure

The project lives inside C:\WordToPDF_Web\wordpdf

```
wordpdf/
├── wordpdf/                    ← Django project config
│   ├── settings.py             ← All configuration (database, apps, paths)
│   ├── urls.py                 ← Root URL dispatcher
│   ├── wsgi.py                 ← WSGI server entry point
│   └── __init__.py             ← MariaDB compatibility patches
│
├── pdfapp/                     ← Main application
│   ├── models.py               ← ConversionHistory database model
│   ├── admin.py                ← Django admin registration
│   ├── apps.py                 ← App configuration
│   ├── migrations/             ← Database schema version history
│   ├── views/                  ← All view handlers
│   │   ├── dashboard.py        ← Core logic: home, history, download
│   │   ├── word_views.py       ← Word tool page routes
│   │   ├── pdf_views.py        ← PDF tool page routes
│   │   ├── excel_views.py      ← Excel tool page routes
│   │   ├── powerpoint_views.py← PowerPoint tool page routes
│   │   └── image_views.py      ← Image tool page routes
│   ├── urls/                   ← URL route definitions
│   ├── services/               ← All conversion business logic
│   │   ├── conversion.py       ← Core file format converters
│   │   ├── compression.py      ← File compression utilities
│   │   ├── editor.py           ← Word & PDF editor services
│   │   ├── pdf_service.py      ← PDF editing and manipulation
│   │   ├── pdf_to_word.py      ← PDF→Word conversion engine
│   │   └── word_com.py         ← Microsoft Word COM automation
│   └── utils/                  ← Helper utilities and validators
│
├── templates/                  ← All HTML templates
│   ├── base.html               ← Master layout template
│   ├── dashboard/home.html     ← Main dashboard page
│   └── includes/               ← Reusable template partials
│       ├── app_scripts.html    ← All JavaScript logic
│       ├── styles.html         ← All CSS styles
│       ├── sidebar.html        ← Sidebar with Recent History
│       └── ...                 ← Other UI partials
│
├── media/uploads/              ← User-uploaded and converted files
├── requirements.txt            ← Python package dependencies
└── manage.py                   ← Django management CLI
```

5. Database Architecture

✓ The project uses MySQL (MariaDB 10.4 via XAMPP) as its only database. SQLite has been permanently removed.

Why MySQL?

- Persistent across browser clears and device changes.
- Shareable across multiple sessions and users.
- Scalable for production deployments.
- Professionally managed via phpMyAdmin.

Connection Details

Setting	Value
Engine	django.db.backends.mysql
Database	docuconverter_db
Host	127.0.0.1
Port	3306
User	root
Password	(blank — XAMPP default)

settings.py — Database Configuration

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME':   'docuconverter_db',
        'USER':   'root',
        'PASSWORD': '',
        'HOST':   '127.0.0.1',
        'PORT':   '3306',
        'OPTIONS': {'charset': 'utf8mb4'},
    }
}
```

MariaDB Compatibility Patch (wordpdf/__init__.py)

XAMPP ships with MariaDB 10.4 but Django 6.0 requires MariaDB 10.6+. A patch in wordpdf/__init__.py bypasses the version check and disables the RETURNING clause so that Django works seamlessly with XAMPP out of the box — no manual upgrade required.

Database Tables

After running migrations, the docuconverter_db contains the essential tables:

Table	Purpose
pdfapp_conversionhistory	Stores every file conversion record with status dropdown mapping.
django_migrations	Tracks which migration scripts have been applied.
auth_user	Stores Django administrator user credentials.

6. Conversion History System

✓ Records are NEVER permanently deleted. Clicking "Clear" or "Delete" only hides records (sets status = "Inactive"). All data is preserved in MySQL forever.

6.1 How It Works

Every successful conversion is automatically saved to the pdfapp_conversionhistory table with status = "Active". The website sidebar only fetches and displays Active records. When a user clears or deletes, the status field is changed to "Inactive" — the row stays in the database.

6.2 The ConversionHistory Model

```
class ConversionHistory(models.Model):
    original_filename  = models.CharField(max_length=255)
    converted_filename = models.CharField(max_length=255)
    conversion_type    = models.CharField(max_length=100)
    file_size          = models.CharField(max_length=50)
    result             = models.CharField(max_length=20, default='Success')
    download_url       = models.CharField(max_length=512)
    created_at         = models.DateTimeField(auto_now_add=True)
    updated_at         = models.DateTimeField(auto_now=True)
    status             = models.CharField(
        max_length=10,
        choices=[('Active','Active'),('Inactive','Inactive')],
        default='Active',
        db_index=True,
    )

    def soft_delete(self):   # Called when user clicks X
        self.status = 'Inactive'
        self.save(update_fields=['status','updated_at'])

    def restore(self):       # Called when restoring a record
        self.status = 'Active'
        self.save(update_fields=['status','updated_at'])
```

6.3 What Happens at Each Action

User Action	What Happens in Database	What User Sees
New Conversion	Record inserted with status = "Active"	Always visible on website
Click X (delete row)	status changed to "Inactive"	Hidden from website. DB row preserved.
Click Clear History	All Active records set to status = "Inactive"	Website shows empty history. DB unchanged.
Restore (API)	status changed back to "Active"	Record reappears on website.
phpMyAdmin	All rows (Active + Inactive) are always visible	Full audit trail always available.

6.4 API Endpoints for History

Method	URL	Action
GET	/history/	Fetch the last 20 Active records as JSON.
POST	/history/delete/<id>/	Soft-delete: marks one record

		Inactive.
POST	/history/restore/<id>/	Restore: marks one record back to Active.
POST	/history/clear/	Soft-clear: marks ALL active records Inactive.

7. Django Configuration Reference

settings.py

Central configuration file. Defines INSTALLED_APPS, database connection, middleware, templates directory, media path, and logging.

urls.py

Root URL dispatcher. Routes all incoming requests to pdfapp.urls, which then splits them by category (pdf/, word/, excel/, etc.). Also hosts the Django admin route.

wordpdf/__init__.py

Loaded before Django starts. Contains the MariaDB 10.4 compatibility patch that bypasses the version check and disables RETURNING clauses.

pdfapp/models.py

Defines the ConversionHistory model with all its fields and the soft_delete() / restore() helper methods.

pdfapp/views/dashboard.py

The core view file. Handles all POST actions (convert, edit, merge, split, compress), file download, and the history API endpoints.

pdfapp/migrations/

Contains the database migration scripts. 0003_rename_status_and_is_active.py implements status dropdown.

templates/includes/app_scripts.html

All JavaScript for the frontend. Handles file uploads, progress bars, history rendering, soft delete, and the clear confirmation dialog.

templates/includes/sidebar.html

The sidebar UI. Contains the Recent History list and the Clear History trash button.

---

8. API Endpoint Reference

Method	URL	Description
GET	/	Home dashboard page.
POST	/	Submit a file operation (convert, merge, split, compress, edit).
GET	/download/<filename>	Download a previously converted file.
GET	/history/	Return last 20 Active conversion records as JSON.
POST	/history/delete/<id>/	Soft-delete one conversion record (marks Inactive).

POST	/history/restore/<id>/	Restore one Inactive record back to Active.
POST	/history/clear/	Mark all Active records as Inactive (with confirmation).
GET	/pdf/	PDF tools category dashboard.
GET	/pdf/pdf-to-word/	PDF to Word tool page.
GET	/pdf/pdf-merge/	PDF Merge tool page.
GET	/pdf/pdf-split/	PDF Split tool page.
GET	/pdf/pdf-compress/	PDF Compress tool page.
GET	/pdf/pdf-to-excel/	PDF to Excel tool page.
GET	/pdf/pdf-edit/	PDF Editor tool page.
GET	/pdf/image-to-pdf/	Image to PDF tool page.
GET	/word/	Word tools category dashboard.
GET	/word/word-to-pdf/	Word to PDF tool page.
GET	/excel/	Excel tools category dashboard.
GET	/powerpoint/	PowerPoint tools category dashboard.
GET	/image/	Image tools category dashboard.

9. Conversion Workflows

9.1 Word to PDF

- 1. User uploads a .docx file via the browser form.
- 2. Django saves it to media/uploads/ using FileSystemStorage.
- 3. services/word_com.py opens the file using win32com.client (Microsoft Word COM).
- 4. Word.Application.SaveAs() exports the document as PDF with format code 17.
- 5. The output PDF path is returned and saved to ConversionHistory.
- 6. User receives a download link in the browser.

9.2 PDF to Word

- 1. User uploads a .pdf file.
- 2. validate_pdf_file() checks for corruption and password protection.
- 3. services/pdf_to_word.py uses pdf2docx to reconstruct tables, images, and paragraphs.
- 4. Output .docx is saved to media/uploads/.
- 5. Record is saved to ConversionHistory with status="Active".
- 6. User downloads the reconstructed Word file.

9.3 Clear History Workflow

- 1. User clicks the trash icon in the sidebar.
- 2. JavaScript shows a confirmation dialog.
- 3. On confirm, JavaScript POSTs to /history/clear/ with a CSRF token.
- 4. Django view runs: ConversionHistory.objects.filter(status="Active").update(status="Inactive")
- 5. NO DELETE SQL is executed — all rows are preserved in the database.
- 6. JavaScript calls renderRecentDownloads() — sidebar now shows "No recent history".
- 7. phpMyAdmin still shows all records with status = "Inactive".

---

10. Command-Line Reference

Task	Command
Start the development server	`py manage.py runserver`
Apply pending database migrations	`py manage.py migrate`
Generate new migration files	`py manage.py makemigrations`
Check project for configuration errors	`py manage.py check`
Install a Python package (in venv)	`c:\WordToPDF_Web\.venv\Scripts\python.exe -m pip install <package>`
Export installed packages	`py -m pip freeze > requirements.txt`
Create MySQL database (XAMPP)	`c:\xampp\mysql\bin\mysql.exe -u root -e "CREATE DATABASE docuconverter_db CHARACTER SET utf8mb4;"`
View all history records (MySQL)	`USE docuconverter_db; SELECT * FROM pdfapp_conversionhistory;`

---

11. Error Troubleshooting Guide

Error: ModuleNotFoundError: No module named MySQLdb

Reason: mysqlclient is not installed in the active Python environment.

Fix: Run: c:\WordToPDF_Web\.venv\Scripts\python.exe -m pip install mysqlclient

Error: NotSupportedError: MariaDB 10.6 or later is required

Reason: Django 6.0 checks the MariaDB version. XAMPP ships with MariaDB 10.4.

Fix: The compatibility patch in wordpdf/__init__.py automatically suppresses this error. Ensure the patch file exists.

Error: KeyError: admin

Reason: Django tried to resolve an admin URL but django.contrib.admin is not in INSTALLED_APPS.

Fix: Ensure admin.site.urls is registered and django.contrib.admin is in settings.py INSTALLED_APPS.

Error: ProgrammingError: RETURNING not supported

Reason: MariaDB 10.4 does not support the RETURNING clause that Django 6.0 uses.

Fix: The patch in wordpdf/__init__.py sets DatabaseFeatures.can_return_columns_from_insert = False automatically.

Error: com_error: CoInitialize has not been called

Reason: Microsoft Word COM was invoked from a background thread without initializing the COM runtime.

Fix: The word_com.py service calls pythoncom.CoInitialize() before dispatching. Ensure this file is not modified.

Error: PermissionError: [Errno 13] on media/ file

Reason: The output file is locked by a running Word process.

Fix: Open Task Manager and kill any stale WINWORD.EXE processes, then retry.

Error: CSRF token missing or incorrect

Reason: File upload form does not include the Django CSRF token.

Fix: Add {% csrf_token %} inside the <form> tag in the template, or pass the X-CSRFToken header in JavaScript fetch requests.

---

12. Developer Interview Q&A

Q: What is COM automation and how is it used here?

A: COM (Component Object Model) is a Windows binary interface standard. DocuConverter Pro uses win32com.client.Dispatch("Word.Application") to programmatically control Microsoft Word — opening DOCX files and exporting them as pixel-perfect PDFs, identical to doing File → Save As → PDF inside Word.

Q: Why does mysqlclient need to be installed in the .venv?

A: Django uses the mysqlclient C extension to connect to MySQL. Because the project runs inside a virtual environment (.venv), packages must be installed there specifically. Installing globally does not affect the virtualenv.

Q: What is soft delete and why use it?

A: Soft delete means records are never removed with a SQL DELETE statement. Instead, a status flag is set to Inactive. This preserves a complete audit trail, allows undo/restore, and protects against accidental data loss.

Q: How does pdf2docx reconstruct a PDF as a Word document?

A: pdf2docx extracts horizontal and vertical line drawings to detect table cell boundaries, then reads text blocks and positions them into Word table cells. Images are extracted as embedded objects. The result is an editable DOCX that mirrors the original PDF layout.

Q: Why was SQLite replaced with MySQL?

A: SQLite stores data in a local file that only one process can write to at a time and is not visible across devices. MySQL is a proper client-server database accessible via phpMyAdmin, supports concurrent connections, and is standard for production Django deployments.

Q: How is the CSRF token handled in JavaScript fetch requests?

A: The CSRF middleware token is embedded in the HTML page as a hidden input. JavaScript reads it with document.querySelector("[name=csrfmiddlewaretoken]").value and sends it as the X-CSRFToken request header in all POST fetch() calls.

13. Coding Standards & Best Practices

Standard	Rule
PEP 8 Compliance	All Python files follow PEP 8 naming and formatting conventions. Use autoflake and pyflakes for automated dead-code removal.
No Hard-Coded Paths	All file paths use BASE_DIR from settings.py or os.path.join() — never absolute strings.
Soft Delete Only	Never call .delete() on ConversionHistory records. Always use .soft_delete() or .update(status="Inactive").
CSRF Protection	Every POST form includes {% csrf_token %}. Every fetch() POST sends the X-CSRFToken header.
COM Resource Cleanup	Always call application.Quit() inside a finally block after using Word COM to prevent WINWORD.EXE from staying open.
Temporary File Cleanup	All upload temp files are tracked in temp_upload_paths[] and deleted in the finally block of each request.
Virtualenv Discipline	Always install packages with the .venv Python, not the system Python. Use .venv\Scripts\python.exe -m pip install.
Error Handling	All conversion service calls are wrapped in try/except. public_error_message() filters internal details before sending to the frontend.
Database Migrations	Never manually edit the database schema. Always add fields through Django models.py → makemigrations → migrate.
No DELETE Queries	The Clear History and Delete buttons must never execute a SQL DELETE. Use UPDATE status="Inactive" only.